Explanation

How the comparison of Dijkstra's algorithm with the two Duan et al. shortest-path algorithms works

Companion guide to the research paper

12 October 2026

Written for the author's own understanding; the paper does not refer to this document.

Contents

1 The project in one page **3**

2 Background: the problem and Dijkstra's algorithm **4**
2.1 Shortest paths, labels and relaxation 4
2.2 Dijkstra's algorithm and the "sorting barrier" 4
2.3 The comparison-addition model 4

3 How the two new algorithms work **4**
3.1 The shared idea: solve bounded sub-problems recursively 4
3.2 The 2025 algorithm (BMSSP-25) 5
3.3 The 2026 algorithm (BMSSP-26): what changes 5
3.4 Two assumptions both papers make 6

4 A worked mini-example of pivot finding **6**

5 The code **7**
5.1 Design decisions worth knowing 7
5.2 From pseudocode line to code 8

6 Three places where the code deliberately differs from the printed pseudocode **8**
6.1 Deleting completed vertices from D (both algorithms) 8
6.2 The boundary returned by the 2026 base case 9
6.3 Re-selecting a representative for an emptied group (2026) 9
6.4 An implementation pitfall that is not in the paper 9

7 How correctness was checked **9**

8 The experiments and how to read the results **10**
8.1 What was measured, and how 10
8.2 The data 10
8.3 Road networks: time (paper Table V; this chart is only in the guide) 11
8.4 Road networks: counts (paper Table VI and Fig. 1) 11
8.5 Validation table (paper Table IV) 12
8.6 Degree reduction (paper Table VII) 12
8.7 Parameter sweep (paper Fig. 2) 12
8.8 Growing graphs: time (paper Fig. 3) 13
8.9 Growing graphs: counts (paper Fig. 4) 13

8.10 The graphs from bzantium's repository (paper Table IX) 14
8.11 Density and weight range (paper Section VI-I; this chart is only in the guide) . . . 15
8.12 Memory (paper Section VI-J) 15

9 Do other implementations agree, and can ours be trusted? 15
9.1 What others found 15
9.2 Is our implementation correct? 16
9.3 Is it unreasonably slow? (Would a better implementation beat Dijkstra?) 16
9.4 Does it have the time complexity of the papers? 17
9.5 Others say "3–4 times slower"; why does one of our graphs show 8 times? 17
9.6 "On large graphs the new algorithms are faster"? 18

10 Reproducing everything 18

11 Redrawing the figures yourself 18

12 What was checked against the proposal and the sources 19

13 How the paper refers to the code, and what is left for you 20

14 Limits of what the study shows 21

15 Glossary 21

1 The project in one page

Question. Dijkstra's algorithm (1959) has been the standard way to compute shortest paths from one source. In 2025 Duan, Mao, Mao, Shu and Yin published an algorithm that is *asymptotically* faster on sparse graphs, $O(m \log^{2/3} n)$ instead of $O(m+n \log n)$, and in 2026 four of them improved it to $O(m\sqrt{\log n} + \sqrt{mn \log n \log\log n})$. Both papers are pure theory. The research paper asks: what do these two algorithms look like next to Dijkstra's algorithm in theory, and how do they behave when all three are actually implemented and run on the same graphs?

What was done.

1. Both new algorithms were implemented in C++ directly from the pseudocode in the two papers (files `scripts/src/bmssp25.hpp` and `scripts/src/bmssp26.hpp`), next to two versions of Dijkstra's algorithm (`dijkstra.hpp`).

2. The implementations were tested against an independent Bellman–Ford computation on 200 000 tiny and 30 000 medium random graphs, and against Dijkstra on every benchmark run. No run ever produced a wrong distance.

3. All algorithms were run on the five real road networks you specified (the SSTD 2005 collection from the `mehdi-ansari` repository) and on four families of generated graphs with up to $2^{22} \approx 4.2$ million vertices.

4. Two kinds of cost were measured: wall-clock time, and *operation counts* (how many comparisons, additions and edge scans each algorithm performs). Counts do not depend on the computer, so they tell us whether a slowdown comes from the algorithm doing more work or from each step being more expensive.

Answer. On every graph tested, Dijkstra's algorithm with an ordinary binary heap was the fastest (between 1.2 and 8.7 times faster). On the road networks it was 3.5 to 8.0 times faster, and there the new algorithms even perform more comparisons and 3–5 times more additions per edge, so they lose in the abstract cost model too. On large random graphs the picture in the abstract model is different: from a few tens of thousands of vertices on, the 2025 algorithm needs *fewer* comparisons than Dijkstra, exactly as the theory predicts, but it is still slower on the clock (1.5 times at four million vertices) because it looks at every edge 3–5 times and each of its steps is more expensive. The 2026 algorithm really does make the "pivot finding" step cheaper, as its authors intended, but it spends the saving elsewhere and was not faster than the 2025 algorithm in practice. While testing, two small inconsistencies in the 2026 pseudocode were found (Section 6); they do not make its answers wrong.

Files.

`research-paper.pdf`	the paper (IEEE conference format, as in `Resources/format.docx`)
`explanation.pdf`	this document
`figure_data.txt`	every number plotted in the paper's figures, as plain text (Section 11)
`paper/main.tex`, `paper/results_rest.tex`	LaTeX source of the paper
`explanation/`	LaTeX source of this guide
`scripts/`	all code (Section 5) and the commands to reproduce (Section 10)
`results/`	every raw measurement as CSV, plus `summary.json`
`data/sstd2005/`	the downloaded road networks

2 Background: the problem and Dijkstra's algorithm

2.1 Shortest paths, labels and relaxation

We are given a directed graph with n vertices and m edges (the paper says "arcs"); every edge (u, v) has a weight $w_{uv} \geq 0$. We want the length $d(v)$ of the shortest path from a source s to every vertex v.

Every algorithm in this project keeps, for each vertex, a *label* $\widehat{d}[v]$: the length of the best path to v found so far. Initially $\widehat{d}[s] = 0$ and all other labels are ∞. The only way a label changes is by *relaxing* an edge (u, v): if $\widehat{d}[u] + w_{uv}$ is smaller than $\widehat{d}[v]$, set $\widehat{d}[v]$ to it. Labels therefore only go down and are never below the true distance. A vertex is called *complete* once $\widehat{d}[v] = d(v)$.

2.2 Dijkstra's algorithm and the "sorting barrier"

Dijkstra's algorithm keeps all labelled but unfinished vertices in a priority queue. It repeatedly takes out the vertex with the smallest label (that vertex is guaranteed to be complete) and relaxes its outgoing edges. Every edge is looked at exactly once.

The catch is the priority queue. Vertices come out in increasing order of distance, so the algorithm sorts the vertices as a side effect, and sorting n numbers by comparisons needs about $n \log n$ comparisons. This is the "sorting barrier". With a Fibonacci heap the total time is $O(m + n \log n)$; with a simple binary heap it is $O((m + n) \log n)$, which is what everybody uses in practice because binary heaps are fast.

In 2024 Haeupler et al. proved that Dijkstra's algorithm is optimal *if the output has to list the vertices in order of distance*. The new algorithms escape this by returning only the distances, never the order.

2.3 The comparison-addition model

The bounds above are stated in a model where the only things one may do with weights are add two of them and compare two of them, each at unit cost. This is why the project counts comparisons and additions: they are the "currency" of the theorems.

3 How the two new algorithms work

A note on names. Neither Duan et al. paper gives its algorithm a name; they only name the recursive procedure inside both, BMSSP. Other authors therefore call the 2025 algorithm simply "BMSSP". Because this project compares two of them, the paper defines its own labels once, in the Introduction: BMSSP-25 is the algorithm of Duan, Mao, Mao, Shu and Yin (STOC 2025) and BMSSP-26 is the algorithm of Duan, Mao, Shu and Yin (ICALP 2026). The paper uses only these two names; this guide also says "the 2025 algorithm" and "the 2026 algorithm" where that reads more easily.

3.1 The shared idea: solve bounded sub-problems recursively

Call the set of labelled-but-unfinished vertices the *frontier*. Dijkstra needs the minimum of the frontier at every step, which forces it to keep the whole frontier ordered. The new algorithms avoid this with two ideas.

Idea 1: bounded multi-source shortest path (BMSSP). Instead of asking for one vertex at a time, ask a bigger question: *given a set S of frontier vertices and a bound B, complete every vertex whose true distance is below B and whose shortest path passes through S.* This question is

answered recursively. A call

$$\text{BMSSP}(\text{level } l,\ \text{bound } B,\ \text{frontier } S)$$

repeatedly (a) takes the roughly M smallest vertices S_i out of its frontier, together with a value B_i that separates them from the rest; (b) calls $\text{BMSSP}(l-1, B_i, S_i)$, which returns a set U_i of now-complete vertices; (c) relaxes all edges leaving U_i and puts the improved neighbours into its frontier. Level 0 is a small Dijkstra search. The whole problem is one call with $S = \{s\}$ and $B = \infty$.

The data structure holding the frontier only has to deliver "the M smallest, in any order", not a sorted sequence. It stores keys in *blocks*; blocks are ordered among each other but unsorted inside. That is cheaper than a heap.

A call may also stop early when it has completed "enough" vertices (a *partial execution*); it then returns a smaller bound $B' < B$ meaning "everything below B' is finished".

Idea 2: shrink the frontier to pivots. If every frontier vertex went through the recursion, the recursion would in the end sort them and nothing would be gained. So each call first reduces its frontier S to a much smaller set, by a factor of about k. The two algorithms differ mainly in how they do this.

3.2 The 2025 algorithm (BMSSP-25)

FindPivots. Starting from S, relax edges outward for k rounds (like k rounds of the Bellman–Ford algorithm), only following labels below B. Let W be all vertices touched.

- If W grew larger than $k \cdot |S|$, the frontier is already small compared with the work to be done; keep all of S as pivots.
- Otherwise look at the edges that are currently "tight" ($\widehat{d}[v] = \widehat{d}[u] + w_{uv}$) inside W. They form a forest. A vertex of S becomes a *pivot* only if it is the root of a tree with at least k vertices. Why is this safe? If the shortest path of some vertex leaves S through a root whose tree has fewer than k vertices, that path has fewer than k edges inside the region and the k rounds have already made the vertex complete.

Since every pivot "owns" at least k vertices of W, there are at most $|W|/k$ pivots.

The data structure D (Lemma 3.3 of the paper) supports Insert (a single key), BatchPrepend (many keys that are all smaller than everything stored) and Pull (remove and return the M smallest keys and a separating value). Internally it is two lists of blocks: one for batch-prepended keys and one for inserted keys, the latter indexed by a balanced search tree over block upper bounds. Blocks that overflow are split at their median, found by a linear-time selection algorithm.

The main loop. Insert the pivots into D. While D is not empty and fewer than $k2^{lt}$ vertices have been completed: Pull S_i, B_i; recurse; relax the edges out of U_i. An improved neighbour whose new label is at least B_i is inserted; one whose label lies between the returned B_i' and B_i is collected and batch-prepended, together with the vertices of S_i that the sub-call did not finish.

Base case. A Dijkstra search from the single vertex in S that stops after $k+1$ vertices.

Parameters. $k = \lfloor \log^{1/3} n \rfloor$, $t = \lfloor \log^{2/3} n \rfloor$, number of levels $\lceil \log n / t \rceil$, block size $M = 2^{(l-1)t}$ at level l. For a million vertices this means $k = 2$, $t = 7$ and three levels.

3.3 The 2026 algorithm (BMSSP-26): what changes

Cheaper pivot finding. The k Bellman–Ford rounds cost about k units of work per vertex. The 2026 paper observes that they compute more than needed. Its FindPivots instead runs, from

each vertex x of S, a tiny Dijkstra search that stops as soon as it has discovered k vertices. Three things can happen:

- the search finds k vertices: the little tree it explored is kept;
- the search runs into a tree kept earlier: the two are glued together and the search stops;
- the search runs out of vertices before reaching k: then everything reachable from x below B has just been finished; x is put aside in a set Q and its vertices in W.

The kept trees form a forest that connects the frontier. It is then cut into pieces of between k and $3k$ vertices (Algorithm 5 of the paper, a depth-first traversal). The frontier vertices in one piece form a group P_j, and only the smallest member p_j of each group is stored in D. When p_j is pulled, the algorithm scans its group and takes along all members below the separator. So D again holds only about a $1/k$ fraction of the frontier, but finding the groups cost only about $\log k$ per vertex.

Simpler data structure. One search tree of blocks. Operations: INSERT, PULL, and MERGE, which absorbs a whole smaller structure whose keys are all below the stored ones. A recursive call now *returns its leftover frontier* as such a structure and the caller merges it; this replaces the BATCHPREPEND bookkeeping of 2025.

Different sizes. Block size $M = t\,2^{(l-1)t}$; a call stops after completing more than $t^3 2^{lt}$ vertices; the base case is a Dijkstra search (on a search tree) that completes up to $t^3 + 1$ vertices and may start from several vertices. Parameters: $t = \lceil \sqrt{\log n \log\log n/\delta}\,\rceil$ and $k = \lceil t/\log t \rceil$, where δ is the degree bound discussed next. For a million vertices: $t = 6$, $k = 3$, four levels, base case of up to 217 vertices.

3.4 Two assumptions both papers make

All path lengths are different. The proofs need that no two paths have the same length (otherwise "the shortest-path tree" is not unique). Real graphs have ties. Both papers therefore compare *tuples* instead of numbers: (length, number of edges on the path, the vertex itself, its predecessor), lexicographically. In the code this is the type `Label` in `common.hpp`. It makes every label four fields wide and every comparison slower than Dijkstra's comparison of two plain numbers.

Small degrees. The running-time proofs assume that every vertex has only a few edges. A graph can be forced into this shape by replacing each vertex by a ring of zero-weight edges, one ring vertex per incident edge. For $\delta = 3$ this roughly doubles-to-quintuples the number of vertices (it becomes $2m$) and triples the number of edges. The code can do this (`reduceDegree` in `graphs.hpp`, option `--reduce`); the main experiments do not, because it makes everything much slower (Table VII of the paper).

4 A worked mini-example of pivot finding

Take $k = 2$, bound $B = \infty$, and suppose the source's edges have been relaxed so that the frontier is $S = \{a, b\}$ with $\widehat{d}[a] = 1$, $\widehat{d}[b] = 4$. Edges: $a \to c$ (weight 2), $c \to d$ (1), $d \to e$ (3); b has no outgoing edges.

2025 FindPivots. Round 1 relaxes the edges out of a and b: c gets label 3. Now $W = \{a, b, c\}$. Round 2 relaxes the edges out of c: d gets label 4. $W = \{a, b, c, d\}$, which is not larger than $k|S| = 4$, so the forest rule applies. Tight edges: $a \to c$, $c \to d$. Trees: $\{a, c, d\}$ rooted at a (3 vertices) and $\{b\}$ (1 vertex). Only a has a tree with at least $k = 2$ vertices, so the pivot set is $P = \{a\}$. Vertex b is dropped from the frontier: it is complete and nothing below B hangs under it. Note that the two rounds cost one relaxation attempt per edge out of a, b, c.

2026 FindPivots. Search from a: discover c (now 2 vertices $= k$), stop; tree $\{a, c\}$ is kept. Search from b: nothing reachable, fewer than k vertices, so b goes to Q and $W = \{b\}$. The forest $\{a, c\}$ is one piece; its only frontier vertex is a, so the single group is $P_1 = \{a\}$ with representative a. Same outcome, but edge $c \to d$ was not touched.

5 The code

Everything is in `scripts/`. The C++ part is header-only plus one driver.

File	What it contains
`src/common.hpp`	Graph in compressed-sparse-row form (`Graph`); the tie-breaking label (`Label`, `lt`); per-vertex label storage (`LabelArray`); the operation counters (`Counters`, macro `CNT`); the worst-case linear selection routine `select_kth`; a small random generator.
`src/dijkstra.hpp`	`DijkstraLazy`: binary heap of (key, vertex) pairs; outdated entries are skipped when popped. `DijkstraIndexed`: heap with positions, supports DecreaseKey.
`src/bmssp25.hpp`	`BlockSeq25`: the Lemma 3.3 structure (`insert`, `batchPrepend`, `pull`, `erase`). `Bmssp25`: `findPivots` (Algorithm 1), `baseCase` (Algorithm 2), `bmssp` (Algorithm 3), `run`.
`src/bmssp26.hpp`	`BlockTree26`: the Lemma 3.4 structure (`insert`, `merge`, `pull`, `erase`, `exportTo`). `BaseSet26`: search tree for the base case. `Bmssp26`: `findPivots` (Algorithm 2 followed by the partition of Algorithm 5), `selectPivot`, `baseCase` (Algorithm 4), `bmssp` (Algorithm 3), `run`.
`src/graphs.hpp`	Loader for the road-network files (`loadSSTD`); generators `genRandom`, `genGrid`, `genBA`, `genHub`; the degree reduction `reduceDegree`.
`src/main.cpp`	Command-line driver with three commands: `info`, `run`, `validate`.
`Makefile`	Builds `bin/sssp` (timing), `bin/sssp_count` (same code with counters, `-DCOUNT_OPS`) and `bin/sssp_debug` (address and undefined-behaviour sanitizers).
`fetch_data.sh`	Clones the data repository at the exact commit used.
`run_experiments.py`	Runs every experiment and writes `results/*.csv`.
`make_figures.py`	Reads the CSV files and writes the figures, the table bodies and `paper/numbers.tex`.
`build_pdfs.sh`	Runs `make_figures.py` and typesets the paper and this guide.

5.1 Design decisions worth knowing

- **Exact arithmetic.** Road lengths in the data have six decimals; they are multiplied by 10^6 and stored as 64-bit integers. Sums are exact, so two algorithms either agree exactly or one is wrong.
- **One label array.** `LabelArray::d[v]` holds (length, hops, predecessor). `through(u,v,w)` builds the candidate label of v via u and counts one addition; `notWorse(c)` checks "candidate $\leq$ current label" and counts one comparison.
- **Sets U without copying.** Every call appends the vertices it completes to one global array `Ubuf` and returns only the index range. A parent's set is automatically the concatenation of its children's ranges.
- **Finding a key in $O(1)$.** For each recursion level there is an array that says in which block and at which position a vertex is stored (a direct-address table). This works because calls on the same level never overlap in time.

- **Stamps instead of clearing.** "Is v in set X?" is answered by comparing `stamp[v]` with a counter that is increased whenever a new set starts, so sets never have to be erased.
- **Selection.** `select_kth` is quickselect; if it has done more than a linear amount of work it switches to the median-of-medians rule, which guarantees linear time in the worst case, as the proofs require.

5.2 From pseudocode line to code

Paper	Code
2025, Alg. 1 lines 4–14 (k relaxation rounds)	`Bmssp25::findPivots`, loop `for (int i = 1; i <= k; i++)`
2025, Alg. 1 lines 15–16 (forest, pivots)	same function, after the loop: `par`, `rootOf`, `tsize`
2025, Alg. 2 (base case)	`Bmssp25::baseCase`
2025, Alg. 3 lines 8–21 (main loop)	`Bmssp25::bmssp`, the `while` loop
2025, Lemma 3.3	class `BlockSeq25`
2026, Alg. 1 (Relax)	`Bmssp26::relax`
2026, Alg. 2 lines 2–18 (local searches)	`Bmssp26::findPivots`, loop over `S`
2026, Alg. 2 line 20 and Alg. 5 (partition)	same function, part "partition the forest"
2026, Alg. 2 lines 21–22 (groups P_j)	same function, last loop
2026, Alg. 3 lines 9–30 (main loop)	`Bmssp26::bmssp`, the `while` loop
2026, Alg. 3 lines 31–38 (finalization)	same function, after the loop
2026, Alg. 4 (base case)	`Bmssp26::baseCase`
2026, Lemma 3.4 and Appendix A.2	classes `BlockTree26` and `BaseSet26`

(Line and lemma numbers are those of the arXiv versions 2504.17033v2 and 2602.07868v2. The published ICALP version of the 2026 paper uses the same line numbers but numbers its lemmas consecutively: Lemma 3.1 there is Lemma 3, Lemma 3.4 is Lemma 6, Lemma 3.7 is Lemma 9 and Lemma 3.8 is Lemma 10. The research paper cites the published numbering.)

6 Three places where the code deliberately differs from the printed pseudocode

Each difference can be switched off on the command line (`--nofix`, `--literal-base`), and the validation counts how often it matters.

6.1 Deleting completed vertices from D (both algorithms)

A key in D keeps the value it had when it was inserted. During a recursive call the label of that vertex can drop, and the vertex can even be completed by the call, while its old entry is still in the caller's D. Later the old entry is pulled and the vertex's already finished subtree is explored again.

For the 2025 algorithm this was found by Castro, Clementino and de Freitas (arXiv:2511.03007); they report that the original authors confirmed the problem and the fix, which is to delete a vertex from D as soon as it is reported complete. The code does that in `Bmssp25::bmssp` (`if (fix) D.erase(u)`).

The 2026 paper says in the proof of its correctness lemma "U_i is not in D, no need to remove". The instrumented runs show otherwise: in 3 475 of the 200 000 tiny graphs and 10 510 of the 30 000 medium graphs a vertex of U_i was still stored in the caller's D when the sub-call returned

(counter `stale_parent`, measured before anything is merged, so it is not an artefact of the merge). Without deletion, vertices get reported complete twice (`dup_u`). The smallest example found has 12 vertices and is saved in `results/validation/witness_2026_duplicate_U.txt`. The code applies the same deletion as for 2025.

How sure is this? The observation is a measurement on my implementation of the pseudocode; I re-read the pseudocode against the code line by line, and the mechanism (keys are not updated when labels drop) is plain from the pseudocode. But the authors have not been asked, so the paper words it as an observation, not as a confirmed erratum. The distances were correct in every run either way.

6.2 The boundary returned by the 2026 base case

Algorithm 4 does, in a loop: pull the smallest vertex u *and a separator* B'; add u to U; relax the edges of u (which inserts new keys). It then returns the last B'. But that B' was computed before the edges of the last vertex were relaxed. If the loop ends because the size limit t^3+1 was reached, the structure can contain keys smaller than the returned B', or be non-empty although $B' = B$ (which is supposed to signal "everything is finished").

Simplest example: a directed path with $t^3 + 2$ vertices, searched from its first vertex. The loop pulls $t^3 + 1$ vertices. The last pull empties the structure, so the separator is B; relaxing the pulled vertex then inserts the final vertex of the path. Returned: $B' = B$ together with a non-empty structure.

The code returns the smallest key left after the loop (`if (baseFix) Bp = base.minOrB()`). The event was seen in 8 939 tiny and 1 435 medium graphs with the printed version; smallest example with 10 vertices in `witness_2026_base_case_bound.txt`.

6.3 Re-selecting a representative for an emptied group (2026)

Line 20 schedules group j for re-selection if its representative was completed and the group is not empty *at that moment*; later vertices of the same U_i can still empty it, and line 28 would then take a minimum over an empty set. The prose of the paper says "for each non-empty P_j". The code follows the prose. (The very first test run crashed here before this was handled.)

6.4 An implementation pitfall that is not in the paper

The sets U are sets. If duplicates produced by the stale-key effect are kept in the list and counted, the top-level loop of the 2025 algorithm ("while $|U| < k2^{lt}$") can end before all vertices are done: an early version of the code that counted duplicates returned wrong distances in 8 of 200 000 tiny graphs when the deletion fix was switched off. The code now removes duplicates when the fix is off (`dropDuplicates`), which is what a set union means. With the fix on, duplicates never occur.

7 How correctness was checked

1. `sssp_count validate` generates random graphs of six shapes (sparse, dense, path with chords, star, tree with extra edges, grid) with five weight ranges, including "all weights zero" where every path ties. Each graph is solved by a simple Bellman–Ford routine (the reference) and by 17 variants: both Dijkstras; BMSSP-25 and BMSSP-26 with the paper's parameters and with random small parameters, each as printed and repaired; and the algorithms on degree-reduced graphs.

2. Two campaigns are stored in `results/validation/`: 200 000 graphs with up to 16 vertices and 30 000 graphs with up to 400 vertices; in total 3 400 000 and 510 000 algorithm runs with 0 and 0 wrong results.

3. `make -C scripts check` repeats a smaller campaign with the address and undefined-behaviour sanitizers switched on (no memory errors).

4. In every timing and counting run the driver first runs Dijkstra and then compares all n distances; the `ok` column of every CSV row is 1, and `make_figures.py` refuses to continue otherwise.

8 The experiments and how to read the results

8.1 What was measured, and how

- **Time.** One "run" is one complete shortest-path computation from one source (start vertex) to *all* vertices, over the whole graph and all its edges. It includes resetting the labels but not loading the graph. The option `--sources N` is the number of different start vertices, not a number of edges: 20 on the five road networks, 10 on the SNAP graphs, 3 on generated graphs. Every algorithm is run from the same sources; for each source the run is repeated (9 times on road networks, 3 on SNAP graphs, 5 on generated graphs) and the median is kept. Results differ little between start vertices on the same graph. To compare two algorithms the ratio of their times is taken *for each source separately*, and the geometric mean of these ratios is reported. A ratio of 4 means "four times slower than Dijkstra".

- **Counts.** A second executable built from the same source code (`bin/sssp_count`) increments a counter at every comparison of two labels or keys (including those inside heaps, search trees and the selection routine), at every addition $\widehat{d}[u] + w_{uv}$, and at every edge scan. All counts are divided by the number of edges m, so "1.0 scans per arc" means every edge was looked at once on average.

- **Baseline.** "Dijkstra" always means the binary-heap version with lazy deletion (`DijkstraLazy`); "Dijkstra (DecreaseKey)" is the indexed-heap variant.

- **Settings.** Unless a table says otherwise, the new algorithms run on the graph as it is (no degree reduction), with the parameters their papers prescribe and with the three repairs of Section 6.

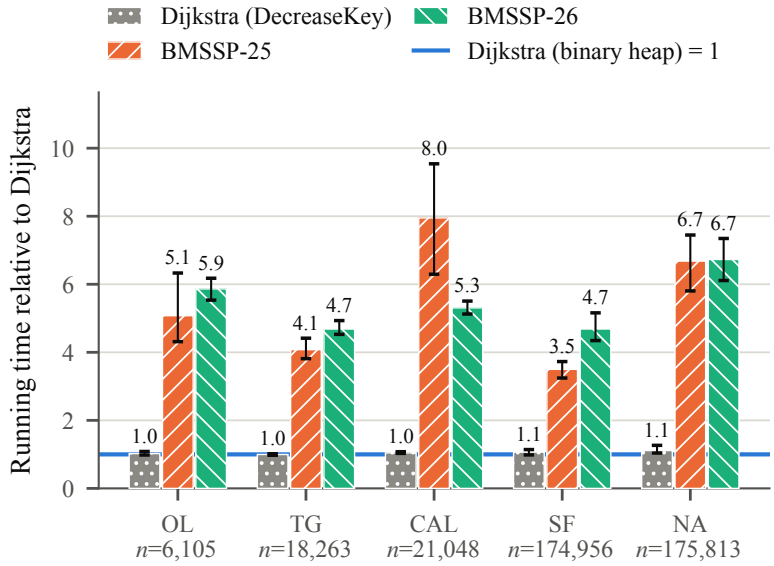
8.2 The data

The five road networks are Oldenburg (OL, 6 105 vertices), San Joaquin County (TG, 18 263), California (CAL, 21 048), San Francisco (SF, 174 956) and North America (NA, 175 813). Each line of an edge file is one road segment with its length. Segments are used in both directions (see Section 12 for why), so the graphs have between 14 070 and 446 002 directed edges. They are very sparse (2.0 to 2.6 edges per vertex) and "thin": when Dijkstra runs on them its heap never holds more than 1.7% of the vertices at once.

Six larger real graphs from the SNAP collection were added after looking at `bzantium/bmssp-python`, which benchmarks on them (Section 8.10). They are downloaded by `sh scripts/fetch_data.sh snap`.

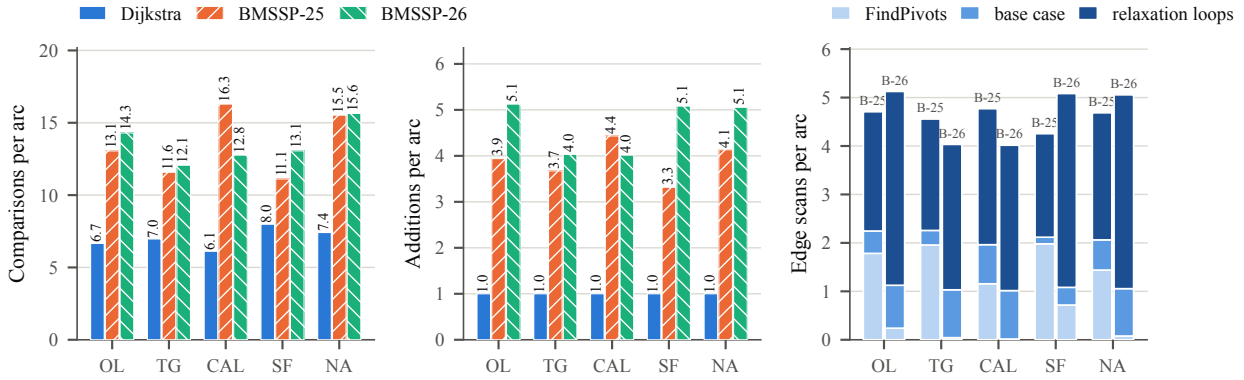
In addition, four families of generated graphs were added to see what happens as graphs grow to millions of vertices: random graphs, grids, "preferential attachment" graphs with a few very high-degree vertices, and "hub" graphs designed so that Dijkstra starts with all n vertices in its heap.

8.3 Road networks: time (paper Table V; this chart is only in the guide)



Each group of bars is one network. The blue line at height 1 is Dijkstra. A bar of height 5 means that algorithm needed five times as long. The thin whiskers show the best and worst source. Reading: BMSSP-25 is 3.5–8.0 times slower and BMSSP-26 is 4.7–6.7 times slower than Dijkstra; the two Dijkstra variants are practically equal. BMSSP-26 beats BMSSP-25 on only one of the five networks.

8.4 Road networks: counts (paper Table VI and Fig. 1)



Left and middle panel (in the paper these numbers are Table VI): comparisons and additions per edge. Dijkstra makes 6–8 comparisons and exactly 1 addition per edge. The new algorithms make 11–16 comparisons and 3–5 additions. So on road networks they are slower *already in the abstract cost model*, before any hardware effects.

Right panel (this is Fig. 1 of the paper): how often each edge is scanned, and in which part of the algorithm. Dijkstra: once. BMSSP-25 (bars marked B-25): 4–5 times, of which more than one whole pass happens inside FindPivots (light blue). BMSSP-26 (B-26): 4–5 times as well, but almost none of it in FindPivots. This is the clearest picture of what the 2026 paper changed: pivot finding became cheap. The dark part (relaxation loops) is larger for 2026 because every recursion level relaxes every edge once and the 2026 parameters give one more level on three of the five networks.

Cost per operation. If an algorithm needs 4.5 times the time but only 2 times the operations, each of its operations costs about 2.3 times as much. This number is 2.1–2.9 for BMSSP-25 and 2.3–2.7 for BMSSP-26. It reflects things the abstract model ignores: bigger labels, bookkeeping,

and memory accesses that the processor cannot predict.

8.5 Validation table (paper Table IV)

Rows: the algorithm as printed in its paper and with the repairs. Columns: number of runs; runs with a wrong distance (always 0); runs in which a stale key was seen; runs in which a vertex was reported complete twice; runs in which the base-case boundary invariant was violated. Reading: the stale keys occur in both algorithms; without the deletion they cause duplicate completions; with the repairs the duplicate and boundary columns are zero.

On the real road networks these effects are rare (duplicates are at most 1.0% of the vertices) and make no measurable difference to the running time; they matter for whether the published proofs apply, not for speed.

8.6 Degree reduction (paper Table VII)

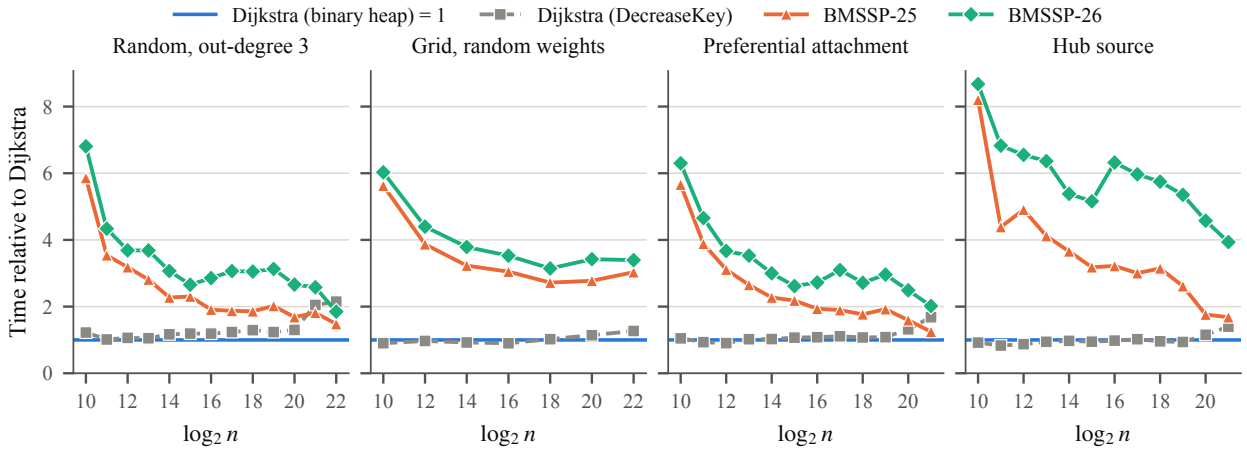
The proofs assume a graph in which every vertex has very few edges. Transforming the road networks into that form makes them 4.1–5.2 times larger, and every algorithm becomes several times slower on the transformed graph. Conclusion: the transformation is needed for the proofs, but in practice one should not apply it. (Castro et al. found the same for the 2025 algorithm.)

8.7 Parameter sweep (paper Fig. 2)



Each cell is one choice of (k, t) on the San Francisco network; the number is the time relative to Dijkstra (smaller and lighter is better). The solid orange box is the choice prescribed by the paper, the dotted box the best one found. Reading: $k = 1$ (bottom row) is bad, which shows that reducing the frontier really is necessary; the prescribed parameters are reasonable; tuning gains 12% (2025) and 11% (2026) on SF and 39% and 31% on NA; but even the best cell is about 3.05 times slower than Dijkstra.

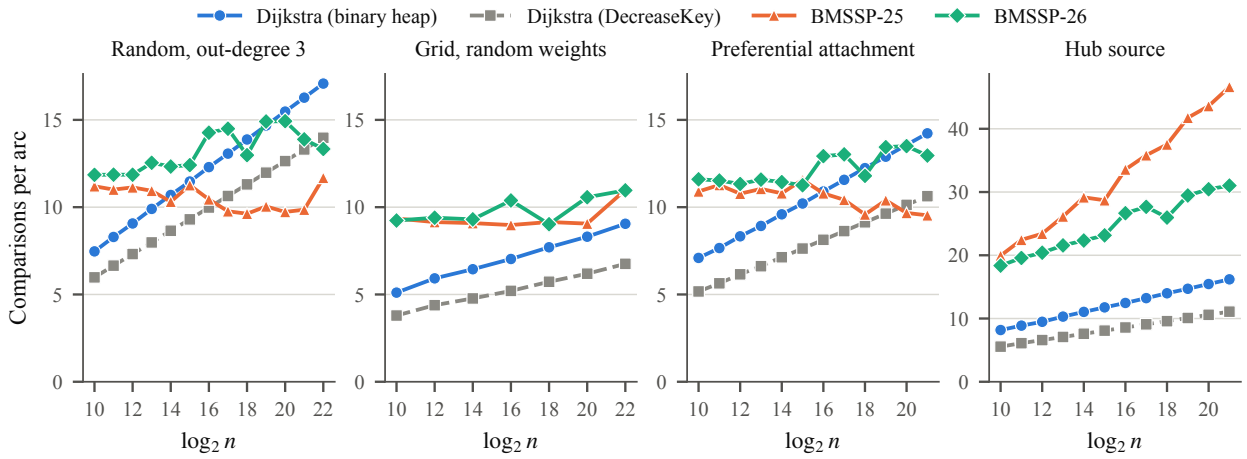
8.8 Growing graphs: time (paper Fig. 3)



Four families of generated graphs, sizes from about a thousand to about four million vertices (the horizontal axis is $\log_2 n$, so each step doubles the size). Vertical axis: time relative to Dijkstra (blue line at 1). Reading:

- *Random graphs and preferential-attachment graphs:* the new algorithms catch up steadily as graphs grow. BMSSP-25 goes from 5.9 times slower at 2^{10} to 1.5 times slower at 2^{22} on random graphs, and to 1.2 on the largest preferential-attachment graph. They never get below the blue line.
- *Grids:* no catching up beyond a certain point; both stay between 2.7 and 3.8 times slower.
- *Hub graphs:* BMSSP-25 reaches 1.7, BMSSP-26 stays at 3.9.
- BMSSP-26 (green) is above BMSSP-25 (orange) in every panel: it was slower on all 44 generated graphs.
- The zig-zags come from rounding: k, t and the number of levels are whole numbers and jump at certain sizes.

8.9 Growing graphs: counts (paper Fig. 4)



This is the most interesting figure: comparisons per edge as the graphs grow. (The additions per edge are in `figure_data.txt` and are quoted in the text.)

- On *random* and *preferential-attachment* graphs the blue Dijkstra line climbs steadily (about 0.80 more comparisons per edge every time the graph doubles: that is the $\log n$ of the sorting

barrier), while the orange BMSSP-25 line stays flat at about 10. The lines cross: from $n = 2^{14}$ (random) and 2^{16} (preferential attachment) on, BMSSP-25 needs fewer comparisons than Dijkstra, and from 2^{16} and 2^{18} on also fewer comparisons plus additions. BMSSP-26 gets below Dijkstra's comparison count at about a million vertices. **So the theoretical advantage is real and already visible at ordinary sizes, if one counts operations.**

- It is still slower on the clock. At 2^{20} random vertices BMSSP-25 performs 11.9 operations per edge against Dijkstra's 16.5, yet needs 1.7 times the time. The abstract model does not charge for looking at every edge 3–5 times, for wider labels or for the bookkeeping of the block structures.
- On *grids* (and road networks) there is no crossing. Reason: Dijkstra's heap stays tiny there (on the largest grid it never holds more than 0.1% of the vertices; on random graphs 33%). With a small heap Dijkstra has almost no sorting work that could be saved.
- On *hub* graphs (note the different vertical scale) the new algorithms do far more comparisons, although the frontier is as wide as possible. Reason: the source has an edge to every other vertex, which breaks the "small degree" assumption. All these edges arrive in the data structure at once and get fully sorted. Paper Table VIII checks this explanation: after the degree-reduction transformation the same graph satisfies the assumption, and then BMSSP-26 needs less than half of Dijkstra's comparisons (55 against 118 per original edge). But the transformed graph is eight times larger, so everything is slower than plain Dijkstra on the original graph (14 comparisons per edge).

Take-away on graph structure. Two properties decide how the new algorithms compare with Dijkstra: how wide Dijkstra's frontier gets (wide is good for the new algorithms), and whether some vertex has a huge degree (bad for them unless the graph is transformed, which is costly).

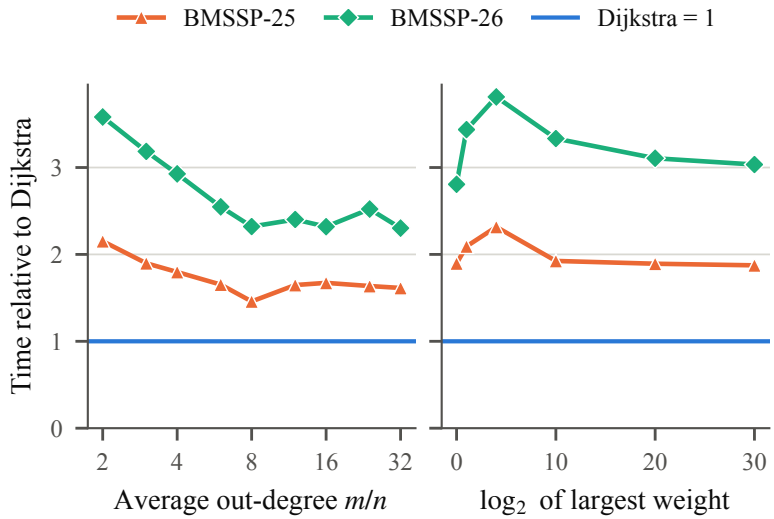
8.10 The graphs from bzantium's repository (paper Table IX)

Six graphs from the SNAP collection, the same ones bzantium's README uses: two web graphs (Stanford, Google), three road networks (Pennsylvania, Texas, California) and the Pokec social network, with 0.3 to 2 million vertices and up to 30.6 million edges. They have no weights, so each is run twice: with all weights 1 (what bzantium does) and with random weights.

Reading of the table: Dijkstra is the fastest on all twelve instances. BMSSP-25 is 1.3–3.2 times slower and BMSSP-26 1.9–4.5 times slower. The advantage that bzantium's table shows for its "V2" solver therefore does not exist for the real algorithm on the very same graphs.

The comparison counts again follow the frontier rule. The three SNAP road networks have narrow frontiers (at most 0.3% of the vertices) and the new algorithms need more comparisons than Dijkstra (1.09–1.43 times for BMSSP-25). The web and social graphs have wide frontiers (13–50%), and there BMSSP-25 needs between 0.79 and 1.65 times Dijkstra's comparisons: fewer than Dijkstra on two of the twelve instances. So the pattern found on generated graphs also shows on real ones.

8.11 Density and weight range (paper Section VI-I; this chart is only in the guide)



Left: random graphs with more and more edges per vertex. The new algorithms get relatively better up to about 8 edges per vertex and then level off (ratios 1.5–2.2 for 2025, 2.3–3.6 for 2026). Right: the size of the weights hardly matters, not even in the extreme case where all weights are 1 and many comparisons are decided by the tie-breaking rule.

8.12 Memory (paper Section VI-J)

On the 4-million-vertex random graph the whole program needed 501 MB with Dijkstra, 859 MB with BMSSP-25 and 1170 MB with BMSSP-26. The extra memory is mainly the per-level lookup tables.

9 Do other implementations agree, and can ours be trusted?

9.1 What others found

Source	What it reports, and what I found when reading it
Castro, Clementino, de Freitas (arXiv:2511.03007; code `lcs147/bmssp`, C++)	Faithful implementation of the 2025 algorithm. Dijkstra is 3–4 times faster on large graphs (up to 27.5 times on small grids). **Agrees with us.**
Valko, Paranjpe, Marx Gómez (arXiv:2509.13448, Rust)	On Lightning Network graphs the 2025 algorithm is consistently slower than Dijkstra. **Agrees with us.**
`bzantium/bmssp-python`, solver "V1" (educational)	Follows the paper; its README says it is 2–4 times slower than Dijkstra. **Agrees with us.**

`bzantium/bmssp-python`, solver "V2" (optimized)	README table shows it beating Dijkstra on Stanford, Google, Pokec, LiveJournal. But the code's own documentation calls V2 a "simplified version" that "eliminates the complex multi-level recursion, pivot finding phases" and the block data structure; it is a bucket queue with fixed bucket width (delta-stepping). The graphs are loaded with *every weight equal to 1*, and one source–target pair is timed. With unit weights a bucket queue is essentially breadth-first search, a linear-time method that only works because the weights are small integers. **It is not the Duan et al. algorithm, so it does not contradict us.**
`alphastrata/DunMaoSSSP` (Rust)	README: "15x faster than Dijkstra on LiveJournal". The benchmark times five random source–target pairs on unit weights against the `petgraph` library routine. In the V2 solver, the recursion returns immediately when pivot finding reaches its size limit (the paper continues with all of S as pivots), and its block structure does not keep blocks ordered. The repository's sanity test uses the 3 353-vertex Rome graph, while V2 answers every graph below 50 000 vertices with a plain Dijkstra fallback. I did not run it; from the source, it is **not a faithful implementation**.
`mfreeman451/bmssp-go`	Claims 1.7–5.5 times speed-up on graphs with at most 2 500 vertices. Same simplified bucket-queue design as bzantium's V2. **Not the Duan et al. algorithm.**

So: every implementation that actually follows the published algorithm finds Dijkstra faster, as we do. The repositories that claim the opposite benchmark a different, simplified algorithm on unit-weight graphs. To be sure, the paper now also runs our implementations on the graphs of bzantium's table (Section 8.10): Dijkstra stays faster there too.

9.2 Is our implementation correct?

Yes, as far as testing can show: 3 400 000 + 510 000 validation runs against an independent Bellman–Ford, and every one of the benchmark runs is compared with Dijkstra on all vertices (Section 7). A wrong implementation would give wrong distances somewhere; ours never did.

9.3 Is it unreasonably slow? (Would a better implementation beat Dijkstra?)

The best available yardstick is Castro et al.'s implementation, because its structure is the same as ours (I compared the two sources function by function: same parameters k, t and levels, same Pull that collects more than M keys from each block sequence, same per-level lookup tables, same deletion fix; the only algorithmic difference is that they track tree roots during the relaxation rounds while we build the forest explicitly as the pseudocode says). Their published times on random graphs with $m = 3n$, next to ours:

	Castro et al. (Intel i5-10400F)			ours (Apple M1)		
n	Dijkstra	BMSSP	ratio	Dijkstra	BMSSP-25	ratio
2^{16}	10.7 ms	38.2 ms	3.6	6 ms	11 ms	1.9
2^{20}	421 ms	1286 ms	3.1	271 ms	454 ms	1.7
2^{22}	2023 ms	7703 ms	3.8	1866 ms	2805 ms	1.5

Our BMSSP is faster than theirs in absolute time and also relative to Dijkstra, so it is not a weak implementation that makes the new algorithm look bad. (Different machines; I did not execute their code, which would mean running third-party code on your laptop. If you want that cross-

check, clone `lcs147/bmssp` and run its `experiments/experiment.cpp` on a graph written by our generator.)

9.4 Does it have the time complexity of the papers?

The honest answer has three parts.

1. **The building blocks have the stated costs.** Selection is worst-case linear; the block index is a red-black tree; finding and deleting a key is $O(1)$ through the per-level tables; Pull, BatchPrepend and Merge touch each key a constant number of times plus the tree operations the papers allow; returning a set costs nothing. Nothing in the code does more than the pseudocode asks for, asymptotically.

2. **Known deviations.** (a) The small local searches use binary heaps where the 2026 paper says Fibonacci heaps; with the tiny heaps involved (at most $k + \delta$ entries) this changes the bound only by a factor that is constant for constant δ. (b) By default the graph is *not* degree-reduced, exactly like Castro et al.'s main variant. The theorems assume bounded degree, so on a graph with a huge-degree vertex the proven bound does not formally apply; the hub experiment (paper Table VIII) shows what then happens, and `--reduce` gives the version to which the theorem applies. (c) The three repairs of Section 6; without the deletion repair the analysis of either paper does not go through.

3. **The measurements behave like the theory.** A time bound cannot be "verified" by experiment, but the counts are consistent with it: on bounded-degree graphs the number of comparisons per edge of both new algorithms stays flat as n grows from 2^{10} to 2^{22}, while Dijkstra's grows by 0.80 per doubling (paper Fig. 4). If our implementation had lost the sub-logarithmic behaviour, its curves would climb like Dijkstra's.

9.5 Others say "3–4 times slower"; why does one of our graphs show 8 times?

Three things explain it, and none of them is a disagreement.

- **The "3–4 times" is for large graphs.** Castro et al.'s own tables show much larger factors on small ones: 25 times at 128 vertices, 5.8 times at 1 024 vertices, up to 27.5 times on small grids. Ours start the same way (paper Fig. 3: about 6 at 1 024 vertices, about 2.3 at 16 384). Beyond that the two differ: their ratio stays between 2.8 and 3.8 all the way to 33 million vertices, while ours keeps drifting down to 1.5 at 4 million. I cannot tell from their published numbers whether that is the implementation or the machine.

- **The 8 is one particular road network (CAL), and it is unusual.** In CAL and NA about 93.5% and 94.8% of all vertices have exactly two neighbours: these files store every bend of a road as a vertex, so the graph is mostly long chains. The other three networks have 17–53%. On the two chain-like networks BMSSP-25 is 6.7–8.0 times slower, on the other three 3.5–5.1 times. Chains are the worst case for the new algorithm relative to Dijkstra: Dijkstra's heap holds almost nothing there, so it is extremely cheap, while the pivot search with $k = 2$ only advances two steps along a chain per round. (With a larger k the gap shrinks: on NA the best setting is 39% faster than the prescribed one.)

- **On ordinary large road networks we get the same as everybody else.** On the three SNAP road networks with 1–2 million vertices BMSSP-25 is 2.6–3.0 times slower than Dijkstra; Castro et al. report an average of 3.7 on the U.S. road networks.

The paper now states this explicitly at the end of Section VI-B.

9.6 "On large graphs the new algorithms are faster"?

No. The paper never says that, and it is not what was measured. What the paper says (abstract, Section VI-F, Fig. 4, conclusion) is narrower: on large random-like graphs the 2025 algorithm performs *fewer comparisons* than Dijkstra; it is still *slower on the clock* on every single graph we ran. "Fewer comparisons" is the quantity the theorem is about; "faster" would be a claim about running time, and for that Dijkstra won everywhere.

10 Reproducing everything

Run from the project folder. The scripts needed are exactly the ones listed.

```
sh scripts/fetch_data.sh snap # 1. download the road networks (+ SNAP graphs)
make -C scripts # 2. build bin/sssp and bin/sssp_count
make -C scripts check # 3. (optional) sanitizer build + validation
python3 scripts/run_experiments.py # 4. all measurements -> results/*.csv
python3 -m venv .venv
.venv/bin/pip install matplotlib numpy
sh scripts/build_pdfs.sh # 5. figures, tables, numbers.tex and both PDFs
```

Step 5 runs `scripts/make_figures.py` (figures, table bodies, `paper/numbers.tex`, `results/summary.json`) and then typesets `paper/main.tex` and `explanation/explanation.tex` with Tectonic, writing `research-paper.pdf` and `explanation.pdf`. Step 4 takes about an hour on the laptop used here (Apple M1, 8 GB). It skips experiments whose CSV already exists; add `--force` to redo them, or name experiments to run only some, for example `python3 scripts/run_experiments.py real params`. The option `--counts-only` redoes only the operation counts and leaves the timings untouched. Timings will differ on another machine; the counts in `*_count.csv` and the validation tables will be identical because all random seeds are fixed.

Experiment	Output files	Used for
`info`	`graph_info.csv`	Table III (road networks)
`validate`	`validation/*.csv`, witness files	Table IV, Section VI-A
`real`	`real_time.csv`, `real_count.csv`	Tables V and VI, Fig. 1
`variants`	`variants_*.csv`	last paragraph of Section VI-A
`reduce`	`reduce_*.csv`	Table VII
`params`	`params_*.csv`	Fig. 2
`scaling`	`scaling_*.csv`	Figs. 3 and 4
`hubreduce`	`hubreduce_*.csv`	Table VIII
`snap`	`snap_*.csv`	Table IX
`density`, `weights`	`density_*.csv`, `weights_*.csv`	Section VI-I
`memory`	`memory.csv`	Section VI-J

No number in the paper's text was typed by hand: `make_figures.py` writes them as LaTeX macros into `paper/numbers.tex`, and the text uses the macros.

11 Redrawing the figures yourself

The paper has four figures. Every value plotted in them is in the plain-text file `figure_data.txt` in the project folder, so you can redraw them in Excel, Google Sheets, Origin, gnuplot or anything

else without touching the code.

- The file is tab-separated. Each block starts with a line `## FIGURE n: ...` that says what the block is, then a few lines `# HOW TO PLOT` with the chart type, what goes on the x axis and the y axis, which columns are the lines or bars, the scales, and any reference line; then a header line with the column names and the rows. Copying the header and rows into a spreadsheet gives one column per field.
- **FIGURE 1** (stacked bars): for each road network and each of the two new algorithms, the edge scans per edge in FindPivots, in the base case and in the relaxation loops. Dijkstra is 1.0 everywhere.
- **FIGURE 2** (heat map): one row per algorithm and k, one column per t; the cell is the time relative to Dijkstra on SF. A second block has the same sweep on NA (quoted in the text).
- **FIGURE 3** (line chart, four panels): family, $\log_2 n$, and the time relative to Dijkstra for the three other programs.
- **FIGURE 4** (line chart, four panels): family, $\log_2 n$, and comparisons per edge for all four programs (additions per edge are included too).
- Blocks marked `EXTRA` hold the numbers of charts that are *not* figures in the paper (the road-network ratios of Table V with the smallest and largest value over the start vertices, the counts of Table VI, and the density and weight sweeps of Section VI-I), in case you want to draw those as well.

The file is rewritten by `scripts/make_figures.py` every time the figures are regenerated, so it always matches the PDF. The complete raw measurements (every single run) are the CSV files in `results/`.

Why four figures and not more: a chart was dropped from the paper when a table or a sentence already gave the same numbers (the bar chart of Table V, the bar charts of Table VI, the density and weight curves, and a second row of the comparison chart). Those charts are still shown in this guide.

12 What was checked against the proposal and the sources

You asked that nothing be assumed. This is what was verified, and how.

- **Both Duan et al. papers were read in full** (arXiv 2504.17033v2; arXiv 2602.07868v2 and the published ICALP PDF). The pseudocode used for the implementation is identical in the arXiv and ICALP versions of the 2026 paper.
- **Proposal error: the degree parameter.** The proposal prints $\delta = 14 \min\{m/n, \log\log n\}$. The paper says $\delta = \frac{1}{4} \min\{m/n, \log\log n\}$ (a lost fraction bar). Moreover this value is below the required minimum 3 for every $n < 2^{4096}$, so in practice $\delta = 3$; the paper states this.
- **Proposal claim "Dijkstra is three to four times faster in every tested scenario"** (about Castro et al.). That is the wording of their abstract, but their own tables report up to 27.5 times on small grids. The paper quotes both.
- **Proposal claim "road networks up to 10^7 vertices".** Castro et al.'s text gives about 2.3×10^7 vertices for the largest road network and 2^{25} for random graphs; the paper uses the latter figure only.

- **Reference [14] of the proposal** has changed its title: arXiv:2509.13448v2 (16 Sep 2026) is called "An Empirical Evaluation of BMSSP and Dijkstra's Algorithm for the Lightning Network". The paper cites v2.
- **Reference [13] of the proposal** lists five authors including L. Castro. The package's own citation and Castro et al.'s bibliography list four (Makowski, Guter, Russell, Saragih); SSRN could not be opened to settle this, so the paper cites the four authors and makes no statement about overlapping authorship.
- **"No peer-reviewed evaluation of the 2026 algorithm exists."** This cannot be proved; the paper says "to our knowledge".
- **Bibliographic data** (authors, venue, pages, DOI) of every journal and conference reference were checked against Crossref or the publisher's page on 12 Oct 2026: STOC 2025 pp. 36–44; ICALP 2026, LIPIcs vol. 374, art. 81, pp. 81:1–81:23; FOCS 2024 pp. 2099–2130; and the classical references.
- **The data set.** The repository's files list each road segment once, always with start id < end id. Read as one-way edges the graphs are almost entirely unreachable (6 vertices reachable from vertex 0 in California), read as two-way they are all connected; so every segment is used in both directions. This is stated in the paper.
- **The earlier AI runs** in the neighbouring folders were read for orientation only. Their claims about the stale keys and the base case were re-derived from my own instrumented implementation; their numbers are not used anywhere.
- **The Dijkstra baseline is not handicapped**: it was timed against `std::priority_queue` and is slightly faster.

13 How the paper refers to the code, and what is left for you

Practice. Papers do not print file paths. The common forms are a sentence such as "the code is publicly available at *URL*" (this is what Castro et al. do) or, in IEEE style, a numbered reference to the software. The IEEE Reference Guide has a "Software" entry type and follows the FORCE11 Software Citation Principles: name the authors, the software, the repository, a version or date and the address. The paper therefore says once, at the end of the implementation section, that implementations, drivers, raw measurements and figure scripts are publicly available, and cites reference [24]:

> Anmol, Tanmay Arora, Adhyyan, and Pahul Veer Singh Gogna. *bmssp-vs-dijkstra: Implementations, experiments and data …* (2026). GitHub. [Online]. Available: `https://github.com/anmoltutejagithub/bmssp-vs-dijkstra`

The road-network data is cited the same way (the SSTD 2005 paper the data owners ask to be cited, plus the GitHub mirror).

To do before submitting.

1. The repository `anmoltutejagithub/bmssp-vs-dijkstra` was created on 12 October 2026 as a *private* repository and contains this folder. Readers of the paper cannot open the link until you make it public (GitHub: Settings → General → Change repository visibility). Do that before you submit.
2. The author block is a single shared block: the four names on one line, then Department of Computer Science & Engineering, Chitkara University, Punjab, India, then the four e-mail addresses in the order of the names. The affiliation wording was taken from the example

block you showed on 12 October 2026; change it at the top of `paper/main.tex` if your department is named differently.

3. Choose a licence for the repository. For long-term citation the usual next step is to archive a release on Zenodo and cite the DOI.

4. Timings come from one laptop. If a reviewer asks, re-run step 4 on a second machine; the counts will not change.

14 Limits of what the study shows

- One machine, one compiler; absolute times are specific to it. The operation counts are not.
- The implementations follow the pseudocode and are reasonably engineered, but they are not tuned to the limit. The counts show that a large part of the gap cannot be engineered away because it is extra work prescribed by the algorithms.
- Weights are integers (an exact subset of the real numbers); floating-point rounding was not studied.
- The largest graphs have 2^{22} vertices because of the 8 GB of memory. Nothing is claimed about where, if anywhere, a crossover would occur.
- The two observations on the 2026 pseudocode are empirical findings on an independent implementation, not statements confirmed by its authors.

15 Glossary

arc a directed edge.

label $\widehat{d}[v]$
best known path length to v; never below the true distance $d(v)$.

relaxation
trying to improve $\widehat{d}[v]$ through an edge (u, v).

complete
a vertex whose label equals its true distance.

frontier
vertices through which all unfinished shortest paths must pass.

BMSSP
bounded multi-source shortest path: the recursive sub-problem of both new algorithms.

pivot a frontier vertex kept for the recursion after the frontier has been thinned out.

k, t the two parameters: k controls how hard pivot finding works, 2^t is the fan-out of the recursion.

δ the degree bound assumed by the 2026 analysis.

partial execution
a call that stops early and returns a bound $B' < B$.

stale key
an entry of the structure D whose stored value is larger than the current label of its vertex.

edge scan
: one look at an edge in any loop of an algorithm (Dijkstra: exactly one per edge).

comparison-addition model
: the cost model of the theorems: only additions and comparisons of weights are counted.